virvelle

CORSO DI MOBILE PROGRAMMING

PROGRAMMA GOL

Virvelle S.r.l. – Società Benefit 2025. All rights reserved. Confidential and proprietary

Finanziato dall'Unione europea
NextGenerationEU

Recap: Dove eravamo rimasti?

Nel Modulo 3 abbiamo affrontato il nemico più grande dello sviluppatore: il **Sistema Operativo**.

- Abbiamo visto come Android uccide le app per liberare RAM.
- Abbiamo capito il **Ciclo di Vita** (la metafora del teatro).
- Abbiamo imparato a salvare i dati dalla "distruzione da rotazione" usando `rememberSaveable`.

Oggi, la nostra app esce finalmente dai confini del telefono e si connette col mondo reale.

virvelle

Obiettivi di Oggi

Asincronia e Rete

1. **Le Basi del Web:** Come viaggiano i dati (IP, DNS, HTTP).
2. **APIs e JSON:** La lingua segreta delle App.
3. **Il Main Thread:** La regola d'oro di Android.
4. **Le Coroutines:** Magia asincrona in Kotlin.
5. **Retrofit & Coil:** Gli strumenti dei professionisti.
6. **Demo:** Connetteremo la nostra prima app a Internet.

virvelle

Parte 1: Fondamenti di Rete

Il viaggio dei Dati

Architettura Client-Server

Le app mobile quasi non fanno veri e propri calcoli complessi (es. l'algoritmo di raccomandazione di TikTok non gira sul vostro telefono).
Il telefono è un **Client**, una "vetrina" che mostra dati elaborati altrove.

- **Client (L'App):** Chiede le informazioni e le disegna belle sullo schermo.
- **Server (Il Cloud):** Il computer potente acceso 24/7 che possiede i dati, fa i calcoli pesanti e fornisce le risposte.

Dibattito: Il Viaggio di Google

Aprite il browser e digitate `www.google.com`. In 200 millisecondi vi appare la pagina.

Cosa succede davvero in quella frazione di secondo dietro le quinte?
Chi sa dirmi come fa il telefono a sapere in quale computer nel mondo, tra miliardi, si trova la pagina di Google?

Indirizzi IP e DNS (La Rubrica Telefonica)

I computer non capiscono i nomi come "google.com", comunicano solo tramite numeri: gli **Indirizzi IP** (es. `142.250.184.46`).

Il **DNS (Domain Name System)** è la rubrica telefonica di Internet.

1. Il telefono chiede al DNS: *"Qual è il numero di google.com?"*
2. Il DNS risponde: *"È 142.250.184.46"*.
3. Il telefono chiama quel numero.

Il Protocollo HTTP (Le regole del gioco)

Una volta trovato il server, Client e Server devono parlare la stessa lingua: **l'HTTP**.

Le due azioni (Metodi) fondamentali che faremo come sviluppatori Android sono:

- **GET:** *"Dammi i dati."* (es. Scarica il meteo, carica il profilo).
- **POST:** *"Prendi questi dati e salvali."* (es. Invia un messaggio su WhatsApp, fai il login con la password).

Che cos'è un'API?

Application Programming Interface

Un server non ti dà accesso libero al suo database (sarebbe un disastro di sicurezza). Crea una porta di servizio chiamata **API**.

Analogia del Ristorante:

- Voi (Client) siete seduti al tavolo.
- La Cucina (Server) ha il cibo (Dati).
- Voi non potete entrare in cucina a cucinare. Parlate con il **Cameriere (l'API)**, gli date l'ordine, e lui vi riporta il piatto pronto.

virvelle

Parte 2: Il Main Thread

Il concetto più importante dello sviluppo Mobile

L'Autostrada dello Schermo: Il Main Thread

In Android, tutto ciò che riguarda la grafica (disegnare un bottone, l'animazione di uno scroll, il click su un testo) avviene su una singola "corsia" autostradale chiamata **Main Thread** (o UI Thread).

Questa corsia lavora a **60 frame al secondo** (o 120 nei telefoni moderni). Significa che deve finire il suo lavoro ogni 16 millisecondi.

La Regola d'Oro Universale

Esiste una regola infrangibile nello sviluppo Android (e iOS):

> **Non eseguire MAI operazioni lunghe o bloccanti sul Main Thread.**

Cosa è un'operazione lunga?

- Leggere un file di testo enorme.
- Salvare dati in un Database locale.
- **Scaricare dati da Internet (il caso di oggi).**

Cosa succede se si infrange la regola?

Se scaricate un'immagine dal web usando il Main Thread, la connessione ci metterà magari 2 secondi.
Durante quei 2 secondi, il Main Thread è **bloccato**. Non può disegnare, non risponde al tocco. L'app si "congela" (Freeze).

Se l'app resta congelata per più di 5 secondi, Android mostra la temutissima schermata **ANR (Application Not Responding)** e chiude l'app!

Torniamo al Ristorante (L'Asincronia)

Immaginate che il cameriere (Main Thread) prenda il vostro ordine, vada in cucina e **si metta lui stesso a cucinare** il vostro piatto per 20 minuti.

Tutti gli altri clienti nel ristorante (le altre parti dell'app) resteranno abbandonati, si arrabbieranno e se ne andranno (L'app crasha).

La Soluzione: Il cameriere lascia l'ordine, *torna in sala a servire altri clienti*, e quando il piatto è pronto viene avvisato dalla cucina! Questa è l'**Asincronia**.

virvelle

Parte 3: Coroutines e JSON

Come si implementa l'Asincronia

Threads vs Coroutines

In passato (con Java) si usavano i *Thread* per l'asincronia, ma erano pesanti e complessi da gestire.

Kotlin ha introdotto le **Coroutines**: sono come dei "Thread super leggeri".
Potete lanciare 100.000 coroutines contemporaneamente e il vostro telefono non farà una piega. Sono perfette per scaricare dati dal web!

La parola magica: `suspend`

Per dire a Kotlin che una funzione è asincrona e potrebbe impiegarci del tempo (come una chiamata di rete), aggiungiamo la parola chiave `suspend` prima di `fun`.

```
// Questa funzione "sospende" l'esecuzione senza bloccare l'app
suspend fun scaricaMeteoDiOggi(): String {
    // Finge di scaricare per 2 secondi
    delay(2000)
    return "Soleggiato, 25°C"
}
```

Lanciare una Coroutine in Compose

Non possiamo chiamare una funzione `suspend` in un punto a caso dell'interfaccia.

Ricordate il **`LaunchedEffect`** visto ieri? È lo spazio sicuro di Compose per le coroutines!

```
var meteo by remember { mutableStateOf("Caricamento...") }

LaunchedEffect(Unit) {
    // Questo codice viene eseguito in una Coroutine asincrona!
    meteo = scaricaMeteoDiOggi()
}

Text(text = meteo)
```

Il Formato dei Dati: Il JSON

Quando facciamo una richiesta HTTP (GET), il server non ci manda un file XML o un file Word. Ci manda un file di testo in formato **JSON** (JavaScript Object Notation).

È universale, leggero ed è letto sia da Android, che da iOS, che dal Web.

Esempio di JSON

Immaginate di scaricare le info di uno studente dal registro elettronico. Ecco cosa vi arriva dal server:

```
{
  "id": 104,
  "nome": "Alfonso",
  "cognome": "Zappia",
  "materie_preferite": ["Informatica", "Sistemi e Reti"],
  "promosso": true
  "hash": "dhjkjorukhvasjlekjrhofgnukvjlrsfdl..."
}
```

Dal JSON a Kotlin (Data Classes)

Come traduciamo quel testo JSON in un oggetto Kotlin utilizzabile? Usando le **Data Class**!

```
// Creiamo lo "stampo" che combacia col JSON
data class Studente(
    val id: Int,
    val nome: String,
    val cognome: String,
    val promosso: Boolean
)
```

Le librerie di Android leggeranno il JSON e riempiranno questa classe automaticamente per noi!

virvelle

Parte 4: Gli Strumenti del Mestiere

Retrofit, Coil e i Permessi

Retrofit (Il Re del Network)

Nessuno in ambito professionale usa le classi base di Android per fare chiamate di rete. L'industria utilizza **Retrofit** (creata da *Square*).

Retrofit prende un'interfaccia Kotlin e genera automaticamente tutto il codice asincrono per fare chiamate HTTP.

Un'interfaccia Retrofit (Esempio)

Basta usare delle annotazioni (`@GET`) per spiegare a Retrofit dove trovare i dati:

Kotlin

```
interface MeteoAPI {
    // Scarica i dati dal sito: [mio-server.com/meteo/roma](https://mio-server.com/meteo/roma)

    @GET("meteo/{citta}")
    suspend fun getMeteoCitta(
        @Path("citta") citta: String
    ): MeteoResponse
}
```

Tutto qui! Al resto penserà la libreria in background.

Coil (Le Immagini in Compose)

Scaricare un testo JSON è facile. Ma come scarichiamo l'immagine di locandina di un film? Serve un'altra libreria per gestire il download, il caching nella memoria e il ritaglio dell'immagine.

Per Jetpack Compose, lo standard industriale di oggi si chiama **Coil**.

Usare Coil in Jetpack Compose

Con Coil, aggiungere un'immagine scaricata da internet diventa facile come scrivere una riga di codice usando il componente `AsyncImage`.

Kotlin

```
AsyncImage(
    model = "[https://mio-sito.com/locandina_film.jpg](https://mio-sito.com/locandina_film.jpg)",
    contentDescription = "Locandina del film",
    modifier = Modifier.size(200.dp)
)
```

Il Guardiano della Rete: Il Manifest

Siamo quasi pronti per scaricare i dati. Ma c'è un dettaglio fondamentale: se provate ad avviare l'app ora, andrà in Crash.

Perché? Perché l'accesso a Internet non è un diritto, è un **Privilegio**. Dobbiamo dichiarare ad Android che la nostra app vuole usare il Wi-Fi o i dati mobili dell'utente.

Aggiungere il Permesso INTERNET

Questo è il primo permesso Android che impareremo a usare. Dobbiamo aprire un file speciale (che si chiama la carta d'identità dell'app) chiamato `AndroidManifest.xml` e aggiungere:

```
<manifest xmlns:android="[http://schemas.android.com/apk/res/android](http://schemas.android.com/apk/res/android)">

    <uses-permission android:name="android.permission.INTERNET" />

    <application>
        ...
    </application>
</manifest>
```

Senza questa singola riga, la vostra app sarà tagliata fuori dal mondo.

virvelle

Parte 5: Architettura dell'App e Stato

I tre stati di una richiesta di Rete

Quando scaricate dei dati, la connessione potrebbe essere veloce (Fibra) o lentissima (Edge in treno). L'interfaccia non può restare bianca mentre aspetta! L'utente penserebbe che l'app si è bloccata.

Dobbiamo **sempre** modellare tre stati distinti nella nostra UI.

Stato 1: Loading ⏳

L'app sta aspettando la risposta. Mostriamo un indicatore di caricamento.

```
if (stato == "LOADING") {
    // La classica rotellina di Android!
    CircularProgressIndicator()
    Text("Ricerca dati in corso...")
}
```

Stato 2: Successo ✅

Il server ha risposto col JSON 200 OK. Nascondiamo la rotellina e disegniamo l'interfaccia reale (la LazyColumn, le immagini, ecc.).

Kotlin

```
if (stato == "SUCCESS") {
    DettagliFilmUi(film = datiScaricati)
}
```

Stato 3: Errore ❌

Il server è giù, oppure l'utente ha spento il Wi-Fi. Non fate crashare l'app! Mostrate un messaggio gentile e un bottone per riprovare.

Kotlin

```
if (stato == "ERROR") {
    Text("Nessuna connessione internet 😢")
    Button(onClick = { riprovaDownload() }) {
        Text("Riprova")
    }
}
```

Dibattito: Progettare per l'Offline

Cosa dovrebbe fare la vostra app se avviata senza connessione?

1. Mostrare una schermata vuota con un errore gigante?
2. Mostrare i dati vecchi scaricati il giorno prima (Cache)?

Come si comportano le app che usate di più (Instagram, TikTok, WhatsApp) quando siete offline in metro?

virvelle

Mettiamoci alla prova!

Kahoot Time 🕹️

Vediamo chi è stato attento sul Main Thread e sull'Asincronia!

Kahoot Time!

PIN: 696660

Breve Demo 💻
(Condivido lo schermo: Andiamo ad interrogare un'API Pubblica (Meteo o Film) e vediamo JSON e Retrofit in azione!)

virvelle

virvelle

Learn Your Way, Grow With Us.

Virvelle S.r.l. – Società Benefit 2025. All rights reserved. Confidential and proprietary